

ICT1011

Computer Organisation and Architecture

Chapter 4

Introduction to Assembly Language Programming

Introduction to Assembly Language Programming

1. Introduction

- What is assembly programming
- Why and when to use assembly programming

2. Assembly Program Basics

- Stages in assembly program development
- Assembly Language Syntax

3. Assembler Directives

- Select assembler sections (.sect, .text, .bss, .usect).
- Define values for memory locations (.byte, .word, .string).
- Create symbol table entries (.equ, .set).
- Define library references and definitions (.global, .ref, .def)
- Specify the end of program (.end)
- Assembly Process

4. Information on Executable Instructions

Introduction to Assembly Language Programming



<https://ict1011.stevenwong.org/c4t1/>

1. Introduction

- What is assembly programming
- Why and when to use assembly programming

2. Assembly Program Basics

- Stages in assembly program development
- Assembly Language Syntax

3. Assembler Directives

- Select assembler sections (.sect, .text, .bss, .usect).
- Define values for memory locations (.byte, .word, .string).
- Create symbol table entries (.equ, .set).
- Define library references and definitions (.global, .ref, .def)
- Specify the end of program (.end)
- Assembly Process

4. Information on Executable Instructions

What is Assembly Language?

High-level Languages

- E.g. C, C++, Java, Basic
- if ($x > 10$)
 $y = 20.2$;
- Easy to write and understand
- Performs data type casting/checking
- Limited or no knowledge needed about the processor
- No details about how fast it would run or how many bytes of memory is needed to store the instructions

What is Assembly Language?

High-level Languages



Assembly Language

- Written in mnemonics of basic instructions supported by the processor

- MOV.B R5, R6*

- MOV.B R6, &temp*

- Not as simple to understand program

- Programmer know exactly what operations the processor is going to perform

- Can calculate the memory needed to store the code and the time needed to execute the code

What is Assembly Language?

High-level Languages



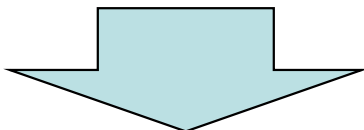
Assembly Language



Machine Code

The exact binary bits that are stored in the memory to instruct the processor to carry out certain operations

494A42162400₁₆



MOV.B R9,R10

MOV.W &2400h, R6

Introduction to Assembly Language Programming



<https://ict1011.stevenwong.org/c4t2/>

1. Introduction

- What is assembly programming
- Why and when to use assembly programming

2. Assembly Program Basics

- Stages in assembly program development
- Assembly Language Syntax

3. Assembler Directives

- Select assembler sections (.sect, .text, .bss, .usect).
- Define values for memory locations (.byte, .word, .string).
- Create symbol table entries (.equ, .set).
- Define library references and definitions (.global, .ref, .def)
- Specify the end of program (.end)
- Assembly Process

4. Information on Executable Instructions

Why Use Assembly Language?

- Very efficient codes can be created:
- Reduce unnecessary programming **overheads**
e.g. high-level language may perform data type checking, but this improvement in code robustness comes at the expense of efficiency.
- More compact program **size**
e.g. embedded devices may not have much built-in memory but require more and more functionalities.
- Codes with faster execution **speed**
e.g. algorithms for processing signals in mobile phone can be computationally demanding and need to operate in real-time.

When to Use Assembly Language?

- Critical parts of the operating system's software
e.g. kernel of system that is constantly being executed.
- Input/Output intensive codes
e.g. device drivers and “loopy” segments of code that processes streaming data (e.g. audio signal enhancement).
- Time-critical or time-dependent codes
e.g. code that detects incoming sensor signals and has to respond quickly (e.g. car braking system).

Introduction to Assembly Language Programming



<https://ict1011.stevenwong.org/c4t3/>

1. Introduction

- What is assembly programming
- Why and when to use assembly programming

2. Assembly Program Basics

- Stages in assembly program development
- Assembly Language Syntax

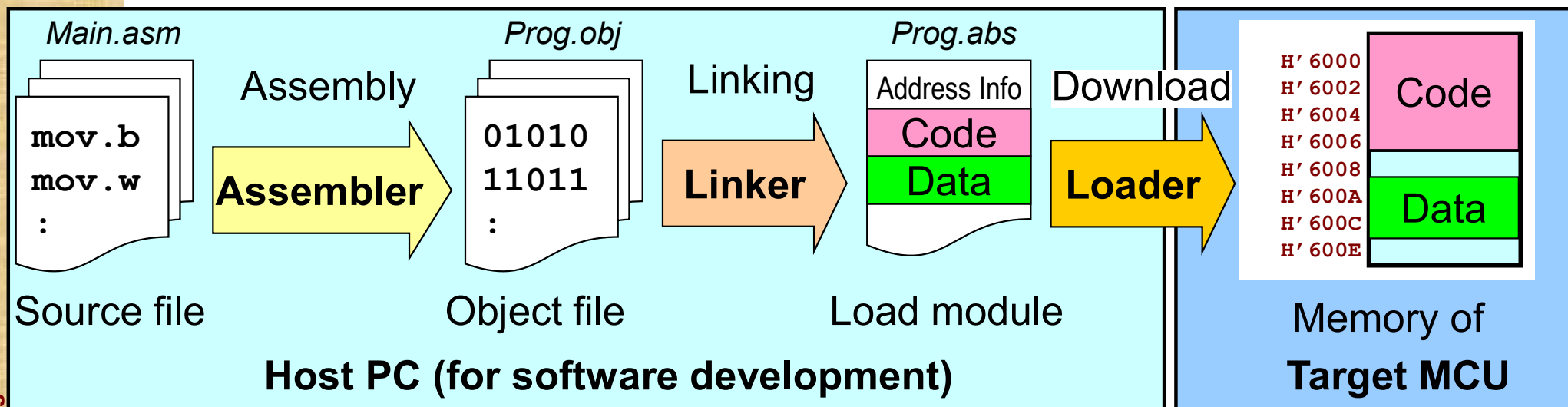
3. Assembler Directives

- Select assembler sections (.sect, .text, .bss, .usect).
- Define values for memory locations (.byte, .word, .string).
- Create symbol table entries (.equ, .set).
- Define library references and definitions (.global, .ref, .def)
- Specify the end of program (.end).A
- Assembly Process

4. Information on Executable Instructions

Assembly Program Development

Stages in developing an assembly program



Assembly Program Development Tools

- **Text editor** – edit the text-based mnemonics in source file (**.asm*).
- **Assembler** – converts mnemonics in source file into machine code and produces an object file (**.obj*).
- **Linker** – combines several object files (e.g. from libraries) into a load module that contains machine code and address information (**.abs*).
- **Loader** – uses load module's address info. to download instructions and data constants into appropriate memory areas for execution.

Characteristics of Assembly Programs

- An assembly language program is made up of basically two types of statements.
- **Executable instructions**
 - these are valid instructions of the processor
 - executed when the program runs
- **Assembler directives**
 - they tell the assembler about the desired characteristics of the program
 - they are processed during program assembly and influence the way the program is loaded into memory.

MSP430 Assembly Program Example

Executable instructions

Assembler Directives

```
.text                                ; Assemble into program memory
.global      RESET
.retain
                                ; Override ELF conditional linking
                                ; and retain current section
.retainrefs                       ; Additionally retain any sections
                                ; that have references to current
                                ; section
```

```
RESET    mov.w    #__STACK_END,SP    ; Initialize stackpointer
StopWDT   mov.w    #WDTPW|WDTHOLD,&WDTCTL ; Stop watchdog timer

Mainloop  bis.b    #001h, &P1DIR      ; Set P1.0 as output
          xor.b    #001h, &P1OUT      ; Toggle value of P1.0 (i.e toggle the LED)
          mov.w    #60000, R15        ; Counter for Delay loop
Delay     dec.w    R15                ; The Delay Loop
          jnz      Delay              ; While R15 is not zero
          ; keep looping back to label Delay
          jmp      Mainloop           ; Jump back to label Mainloop
```

```
.global      __STACK_END
.sect        .stack
.sect        ".reset"                ; MSP430 RESET Vector
.short      RESET
```

Introduction to Assembly Language Programming



<https://ict1011.stevenwong.org/c4t4/>

1. Introduction

- What is assembly programming
- Why and when to use assembly programming

2. Assembly Program Basics

- Stages in assembly program development
- Assembly Language Syntax

3. Assembler Directives

- Select assembler sections (.sect, .text, .bss, .usect).
- Define values for memory locations (.byte, .word, .string).
- Create symbol table entries (.equ, .set).
- Define library references and definitions (.global, .ref, .def)
- Specify the end of program (.end)
- Assembly Process

4. Information on Executable Instructions

Assembly Language Syntax

- The typical assembly language program format divides into four columns.

Label	Mnemonic	Operand	Comments
Delay	dec.w	R15	; The Delay Loop
	jnz	Delay	; While R15 is not zero
			; keep looping back to label Delay
	jmp	Mainloop	; Jump back to label Mainloop
	.global	__STACK_END	
	.sect	.stack	
	.sect	".reset"	; MSP430 RESET Vector
	.short	RESET	

[*label* [:]] *mnemonic* [*operand list*] [;*comment*]

Assembly Language Syntax

- The MSP430 assembler reads up to 200 characters per line. Any characters beyond 200 are truncated.
- Follow these guidelines:
 - All statements must begin with a label, a blank, an asterisk, or a semicolon
 - Labels are optional; if used, they must begin in column 1
 - One or more blanks must separate each field. Tab and space characters are blanks.
 - Comments are optional. Comments that begin in column 1 can begin with an asterisk or a semicolon (* or ;), but comments that begin in any other column must begin with a semicolon.
 - A mnemonic cannot begin in column 1 or it will be interpreted as a label

[*label* [:]] *mnemonic* [*operand list*] [;*comment*]

Assembly Language Syntax

Label	Mnemonic	Operand	Comments
-------	----------	---------	----------

- **Label Field**

- Labels are optional for all assembly language instructions and for most (but not all) assembler directives
- When used, a label must begin in column 1
- A label can contain up to 128 alphanumeric characters (A-Z, a-z, 0-9, _, and \$)
- Labels are case sensitive and the first character cannot be a number
- A label can be followed by a colon (:)
 - A label on a line by itself is a valid statement
- If you do not use a label, the first character position must contain a blank, a semicolon, or an asterisk

Assembly Language Syntax

Label	Mnemonic	Operand	Comments
-------	----------	---------	----------

Address and Value Labels

- It is used as a reference to the address of an instruction or data (**address label**).
- However, when used with the **.equ** directive, it takes on the value of the equated constant (**value label**).

Example

Fri	.equ	5	;Value label "Fri" has the value of 5
	:		
Week	.byte	7	;Address labels "Week" and "Main" are
	:		;assigned the addresses of the memory
Main:	mov.b	R5, R6	;locations where the word-sized data 7
	mov.w	&Week, R7	;and the mov.b instruction are stored
	:		;respectively.

Assembly Language Syntax

Label	Mnemonic	Operand	Comments
-------	----------	---------	----------

- **Mnemonic Field**

- The mnemonic field follows the label field.
- The mnemonic field cannot start in column 1; if it does, it is interpreted as a label.
- The mnemonic field can begin with one of the following items:
 - Machine-instruction mnemonic (such as ADD, MOV, JMP)
 - Assembler directive (such as .data, .list, .equ)
 - *Macro directive (such as .macro, .var, .mexit)*
 - *Macro call*

(We will not cover this in this module)

Assembly Language Syntax

Label	Mnemonic	Operand	Comments
-------	----------	---------	----------

- Contains **mnemonic** of **executable instructions** or **assembler directives**.

- Many opcode mnemonics require a suffix to indicate size of the data being handled.

e.g. – the 2 valid size suffixes of the move mnemonic are

`mov .b`, `mov .w`

- Some examples of assembler directives :

	.data		
Day	.byte	Fri	;initialize byte-sized memory to Fri
Mth	.space	12	;allocate 12 bytes of memory

Assembly Language Syntax

Label	Mnemonic	Operand	Comments
-------	----------	---------	----------

- **Operand Field**

- The operand field follows the mnemonic field and contains one or more operands.
- The operand field is not required for all instructions or directives.
- An operand consists of the following items:
 - Symbols
 - Constants
 - Expressions (combination of constants and symbols)
- You must separate operands with commas

Assembly Language Syntax

Label	Mnemonic	Operand	Comments
-------	----------	---------	----------

- Number of operands in this column depends on the instruction (may be 0, 1 or 2) .

Two-operand example:

mov.b	R5,R6	;move byte in register R6 to R6	
--------------	--------------	---------------------------------	--

One-operand example:

dec.w	R15	;decrement the value in R15	
--------------	------------	-----------------------------	--

No operand example:

ret	;return from subroutine.		
------------	--------------------------	--	--

Assembly Language Syntax

Label	Mnemonic	Operand	Comments
-------	----------	---------	----------

- **Constants**

- Constants are treated as unsigned (FFh is equal to $00FF_{16}$ or 255_{10})
- Types of constants:
 - **Decimal:** string of decimal digits ranging from (-2147483648 to 4294967295 (ie, 1000, -32768)
 - **Hexadecimal:** string of up to 8 hexadecimal digits followed by suffix 'H' (or 'h') or preceded by '0x' (ie, 78h, 0x78)
 - Note: for hex numbers beginning with 'A' – 'F', always precede the number with a '0' else the assembler will treat it as a label.
 - For example: 0A123H is a hex number, A123H is a Label
 - **Octal:** string of up to 8 hexadecimal digits followed by suffix 'Q' (or 'q') or preceded by '0' (ie, 078, 78Q)
 - **Binary:** string of up to 32 binary digits followed by suffix B (or b) (ie. 0000b, 11110000B)

Assembly Language Syntax

Label	Mnemonic	Operand	Comments
-------	----------	---------	----------

• Expressions

- An expression is a constant, a symbol, or a series of constants and symbols separated by arithmetic operators. For example:

mov.w #1+2, R5

- Three main factors influence the order of expression evaluation by the assembler:

- Parentheses Expressions enclosed in parentheses are always evaluated first.
 - $8 / (4 / 2) = 4$
- **Precedence groups Operators:** when parentheses do not determine the order of expression evaluation, the highest precedence operation is evaluated first.
 - $8 + 4 / 2 = 10$ ($4 / 2$ is evaluated first)
- **Left-to-right evaluation:** when parentheses and precedence groups do not determine the order of expression evaluation, the expressions are evaluated from left to right
 - $8 / 4 / 2 = 1$

Assembly Language Syntax

Label	Mnemonic	Operand	Comments
-------	----------	---------	----------

- Operators can be used in expressions and are evaluated according to precedence group

<u>Group</u>	<u>Operator</u>	<u>Description</u>
1	+, -, ~, !	Unary plus, minus, 1's complement, Logical NOT
2	*, /, %	Multiplication, Division, Modulo
3	+, -	Addition, Subtraction
4	<<, >>	Shift left, Shift right
5	<, <=, >, >=	Less than, Less than or equal to, Greater than, Greater than or equal to
6	=[=], !=	Equal to, Not equal to
7	&	Bitwise AND
8	^	Bitwise exclusive OR (XOR)
9		Bitwise OR

Assembly Language Syntax

Label	Mnemonic	Operand	Comments
-------	----------	---------	----------

Comments are ignored by the assembler but are essential for good program documentation.

Use a “;” character before starting comments.

Main:	mov.b	R5, R6
	mov.w	&Week, R7
	mov.w	R12, &Mth

;move byte in register R6 to R6
;move word at memory variable Week to R7
;move word in ER12 to memory var. Mth

Introduction to Assembly Language Programming



1. Introduction

- What is assembly programming
- Why and when to use assembly programming

2. Assembly Program Basics

- Stages in assembly program development
- Assembly Language Syntax

3. Assembler Directives

- Select assembler sections (.sect, .text, .bss, .usect).
- Define values for memory locations (.byte, .word, .string).
- Create symbol table entries (.equ, .set).
- Define library references and definitions (.global, .ref, .def)
- Specify the end of program (.end)
- Assembly Process

4. Information on Executable Instructions

Assembler Directives

- Assembler directives supply data to the program and control the assembly process
- Assembler directives enable you to:
 - Assemble code and data into specified sections
 - Reserve space in memory for uninitialized variables
 - Control the appearance of listings
 - Initialize memory
 - Define global variables
 - Specify libraries from which the assembler can obtain macros
 - many more...

Assembly Directives

Symbols

- Symbols are used as labels, constants, and substitution symbols
- A symbol name is a string of up to 200 alphanumeric characters (A-Z, a-z, 0-9, \$, and _)
- A symbol cannot contain embedded blanks and the first character cannot be a number
- Symbols are case sensitive
- Symbols used as labels become symbolic addresses that are associated with locations in the program
- Labels used locally within a file must be unique.

Commonly Used Directives

- Assembly directives are used to:
 - Select assembler sections (.sect, .text, .bss, .usect).
 - Define values for memory locations (.byte, .word, .string, .space).
 - Create symbol table entries (.equ, .set).
 - Define library references and definitions (.global, .ref, .def)
 - Specify the end of program (.end).

Directives

```

;*****
;   Example Code
;*****
.cdecls      C,"msp430x22x4.h"           ; include C header

COUNT       .equ          2000

;-----
.bss          cnt,2                      ; ISR counter
;-----
.text
start:        mov.w          #0x0400,SP           ; Program reset
               mov.w          #WDT_MDLY_0_5,&WDTCTL ; Initialize stack pointer
               bis.w          #LPM0+GIE,SR        ; Set Watchdog interval to ~0.5ms
               jmp            $                   ; Enter LPM0 w/ interrupt
               ; Loop forever; interrupts do all

               .sect          ".reset"           ; MSP430 RESET Vector
               .word          start              ; Power Up ISR
               end


```

Current Location Counter

Introduction to Assembly Language Programming



<https://ict1011.stevenwong.org/c4t6/>

1. Introduction

- What is assembly programming
- Why and when to use assembly programming

2. Assembly Program Basics

- Stages in assembly program development
- Assembly Language Syntax

3. Assembler Directives

- Select assembler sections (.sect, .text, .bss, .usect).
- Define values for memory locations (.byte, .word, .string).
- Create symbol table entries (.equ, .set).
- Define library references and definitions (.global, .ref, .def)
- Specify the end of program (.end)
- Assembly Process

4. Information on Executable Instructions

Assembly Directives

Select Assembler Sections

- A section is a block of code or data that occupies contiguous space in the memory map.
- Each section has its own Location Counter.
- The assembler assembles into the current section.

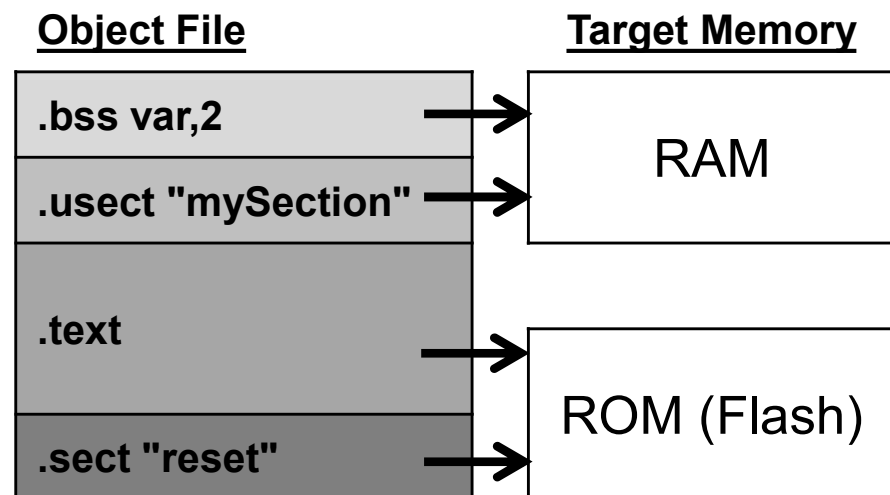
There are two types of sections:

- Initialized sections containing data or code (modal)

- `.sect`
- `.text`
- `.data`

- Uninitialized sections reserving space in the memory map for uninitialized data (temporary)

- `.bss`
- `.usect`



example

Select Assembler Sections

- Assembler directives that define sections:

Mnemonic and Syntax	Description
.bss <i>symbol, size in bytes[, alignment]</i>	Reserves <i>size</i> bytes in the .bss (uninitialized data) section
.data	Assembles into the .data (initialized data) section
.sect " <i>section name</i> "	Assembles into a named (initialized) section
.text	Assembles into the .text (executable code) section
<i>symbol</i> .usect " <i>section name</i> ", <i>size in bytes</i> [, <i>alignment</i>]	Reserves <i>size</i> bytes in a named (uninitialized) section

.text identifies portions of code in the .text section. The .text section usually contains executable code.

.data identifies portions of code in the .data section. The .data section usually contains initialized data.

.sect defines initialized named sections and associates subsequent code or data with that section. A section defined with .sect can contain code or data.

.bss reserves space in the .bss section for uninitialized variables.

.usect reserves space in an uninitialized named section. The .usect directive is similar to the .bss directive, but it allows you to reserve space separately from the .bss section.

example

Select Assembler Sections

Location Counter

- The Location Counter holds the relative memory position of an instruction within the current section.
 - Each section has a location counter used to assign storage addresses to your program's statements.
 - As the instructions of a source module are being assembled, the location counter keeps track of the current location in storage.
 - A \$ (dollar sign) can be used as an operand to an instruction to refer to the current value of the location counter.
- The assembler assembles into the current section.
 - An initialized section directive (**.data**, **.sect**, **.text**) instructs the assembler to stop assembling in the current section and begin assembling in the indicated section.
 - An uninitialized section directive does not end the current section, but simply escape from the current section temporarily. (Thus uninitialized directives **.bss** and **.usect** can appear anywhere in an initialized section without affecting its contents.)

example

Introduction to Assembly Language Programming



1. Introduction

- What is assembly programming
- Why and when to use assembly programming

2. Assembly Program Basics

- Stages in assembly program development
- Assembly Language Syntax

3. Assembler Directives

- Select assembler sections (.sect, .text, .bss, .usect).
- Define values for memory locations (.byte, .word, .string).
- Create symbol table entries (.equ, .set).
- Define library references and definitions (.global, .ref, .def)
- Specify the end of program (.end)
- Assembly Process

4. Information on Executable Instructions

Directives that Initialises Memory

Mnemonic and Syntax	Description
.byte <i>value₁</i> [, ... , <i>value_n</i>]	Initialize one or more successive bytes in the current section
.double <i>floating point value</i>	Initialize a 48-bit MSP430 floating-point constant
.float <i>floating point value</i>	Initialize a 32-bit, MSP430 floating-point constant
.space <i>size in bytes</i>	Reserve <i>size</i> bytes in the current section; note that a label points to the beginning of the reserved space
.string " <i>string₁</i> " [, ... , " <i>string_n</i> "]	Initialize one or more text strings
.word <i>value₁</i> [, ... , <i>value_n</i>]	Initialize one or more 16-bit integers

Some Examples

Directives that Initialises Memory

- .byte** places one or more 8-bit values into consecutive bytes of the current section.
- .word** places one or more 16-bit values into consecutive bytes in the current section.
- .string** places 8-bit characters from one or more character strings into the current section. This directive is identical to .byte.
- .float** calculates a (32-bit) MSP430 floating-point representation of a single precision floating-point value and stores it in four consecutive bytes in the current section.
- .double** calculates a (48 bit) MSP430 floating-point representation of a double precision floating-point value and stores it in six consecutive bytes in the current section.

	<u>Addr</u>	<u>Contents</u>
1	0000	0xaa
	0001	0xbb
2	0002	0x1234
3	0004	0x68
	0005	0x65
	0006	0x6c
	0007	0x70
4	0008	0x81490fdb
5	000C	0x81490fdaa292

AN EXAMPLE

```
.byte  0AAh, 0BBh

.word  01234h
.string "help"

.float 3.141592654
.double 3.141592654
```

Directives that Initialises Memory

The **.space** directive reserves a specified number of bytes in the current section. The assembler fills these reserved bytes with 0s.

When you use a label with **.space**, it points to the first byte of the reserved block.

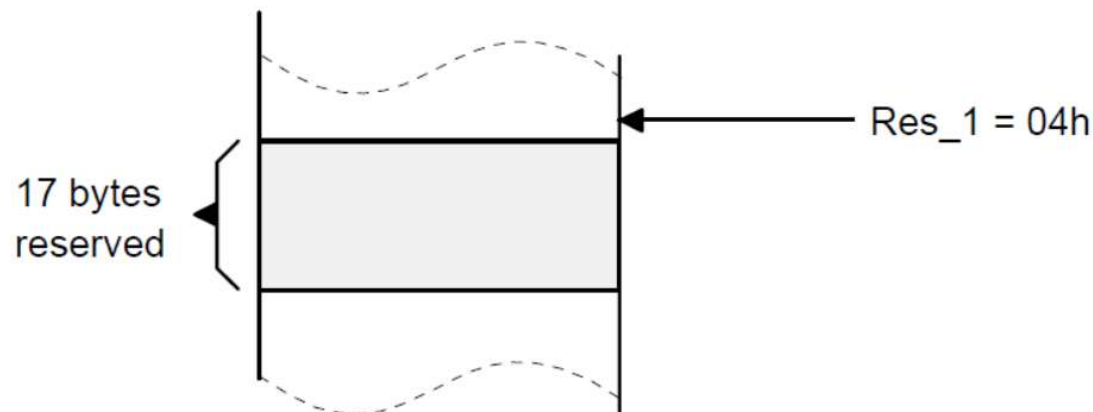
	<u>Addr</u>	<u>Contents</u>
1	0x0000	0x100
	0x0002	0x200
2	0x0004	
3	0x0016	0x000f

AN EXAMPLE

```

                .word 100h,200h

Res_1:  .space 17
                .word 15
    
```



Assembly Directives

Worked Example

Location Counters

Content

1			;*****
2			;* Start assembling into .text *
3			;*****
4	0000		.text
5	0000	01	.byte 1,2
	0001	02	
6	0002	03	.byte 3,4
	0003	04	
7			
8			;*****
9			;* Start assembling into .data *
10			;*****
11	0000		.data
12	0000	09	.byte 9,10
	0001	0a	
13	0002	0b	.byte 11,12
	0003	0c	
14			
15			;*****
16			;* Start assembling into named *
17			;* section, var_defs *
18			;*****
19	0000		.sect "var_defs"
20	0000	11	.byte 17,18
	0001	12	
21			

Assembly Directives

Worked Example

```

22      ;*****
23      ;* Resume assembling into .data      *
24      ;*****
25 0004      .data
26 0004      .byte 13,14
   0005      0e
27 0000      .bss sym,19      ; reserve space in .bss
28 0006      .byte 15,16      ; still in .data
   0007      10
29
30      ;*****
31      ;* Resume assembling into .text      *
32      ;*****
33 0004      .text
34 0004      .byte 5,6
   0005      06
35 0000      usym .usect "xy",20 ; reserve space in xy
36 0006      .byte 7,8      ; still in .text
   0007      08

```

Introduction to Assembly Language Programming



1. Introduction

- What is assembly programming
- Why and when to use assembly programming

2. Assembly Program Basics

- Stages in assembly program development
- Assembly Language Syntax

3. Assembler Directives

- Select assembler sections (.sect, .text, .bss, .usect).
- Define values for memory locations (.byte, .word, .string).
- Create symbol table entries (.equ, .set).
- Define library references and definitions (.global, .ref, .def)
- Specify the end of program (.end).A
- Assembly Process

4. Information on Executable Instructions

Create Symbol Table Entries

- **Syntax** symbol .set value
 symbol .equ value
- **Description**

The **.set** and **.equ** directives equate a constant value to a symbol. The symbol can then be used in place of a value in assembly source. This allows you to equate meaningful names with constants and other values. The **.set** and **.equ** directives are identical and can be used interchangeably.

```

1          ;*****
2          ;* Equate symbol FP to register R11  *
3          ;* and use it instead of R11        *
4          ;*****
5          0b          FP      .set      R11
6 0000      4b24      MOV      @FP, R4
7
8          ;*****
9          ;* Set symbol count to an integer  *
10         ;* expression and use it as an     *
11         ;* immediate operand               *
12         ;*****
13         35          count    .equ      100/2+3
14 0002      40340035      MOV      #count, R4
15

```

It is like a text-replacement tool!!!

Introduction to Assembly Language Programming



<https://ict1011.stevenwong.org/c4t9/>

1. Introduction

- What is assembly programming
- Why and when to use assembly programming

2. Assembly Program Basics

- Stages in assembly program development
- Assembly Language Syntax

3. Assembler Directives

- Select assembler sections (.sect, .text, .bss, .usect).
- Define values for memory locations (.byte, .word, .string).
- Create symbol table entries (.equ, .set).
- Define library references and definitions (.global, .ref, .def)
- Specify the end of program (.end)
- Assembly Process

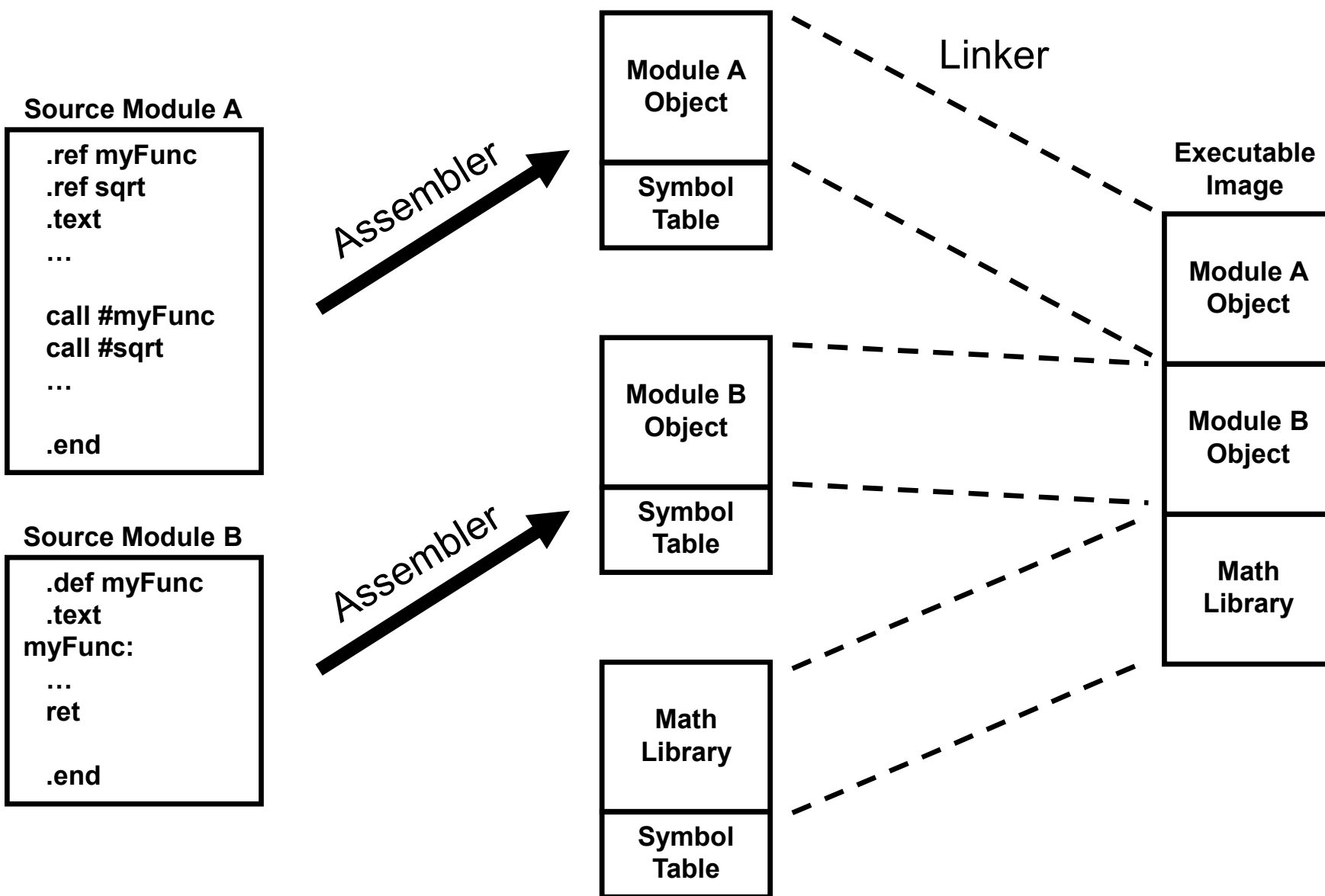
4. Information on Executable Instructions

Library Reference and Definition

- Library
 - A set of routines for a specific domain application.
 - Example: math, graphics, GUI, etc.
 - Use the **.ref** directive to reference symbols defined outside a program.
- Library routine invocation
 - Labels for the routines are defined as **.def**
 - Each library routine contains its own symbol table.
 - A linker resolves the external addresses before creating the executable image.
 - Reports and unresolved symbols.
- The **.global** directive declares a symbol to be external so that it is available to other modules at link time. This directive does double duty, acting as a **.def** for *defined symbols* and as a **.ref** for *undefined symbols*.

Assembly Directives

Library Reference and Definition



Introduction to Assembly Language Programming



<https://ict1011.stevenwong.org/c4t10/>

1. Introduction

- What is assembly programming
- Why and when to use assembly programming

2. Assembly Program Basics

- Stages in assembly program development
- Assembly Language Syntax

3. Assembler Directives

- Select assembler sections (.sect, .text, .bss, .usect).
- Define values for memory locations (.byte, .word, .string).
- Create symbol table entries (.equ, .set).
- Define library references and definitions (.global, .ref, .def)
- Specify the end of program (.end)
- Assembly Process

4. Information on Executable Instructions

Specify End of Program

- **Syntax** `.end`

- **Description**

The ***.end*** directive is an optional directive that terminates assembly. It should be the last source statement of a program. The assembler ignores any source statements that follow an ***.end*** directive.

This directive has the same effect as an end-of-file. You can also use ***.end*** when you're debugging code and you'd like to stop assembling at a specific point in your code.

1	0000		Text_start:	.text
2	0000	0a		.byte 0ah
3	0002	aaaa		.word 0aaaah
4	0004	61		.string "aaa"
	0005	61		
	0006	61		
5				.end

Introduction to Assembly Language Programming



<https://ict1011.stevenwong.org/c4t11/>

1. Introduction

- What is assembly programming
- Why and when to use assembly programming

2. Assembly Program Basics

- Stages in assembly program development
- Assembly Language Syntax

3. Assembler Directives

- Select assembler sections (.sect, .text, .bss, .usect).
- Define values for memory locations (.byte, .word, .string).
- Create symbol table entries (.equ, .set).
- Define library references and definitions (.global, .ref, .def)
- Specify the end of program (.end)
- Assembly Process

4. Information on Executable Instructions

Assembly Directives

Assembly Process

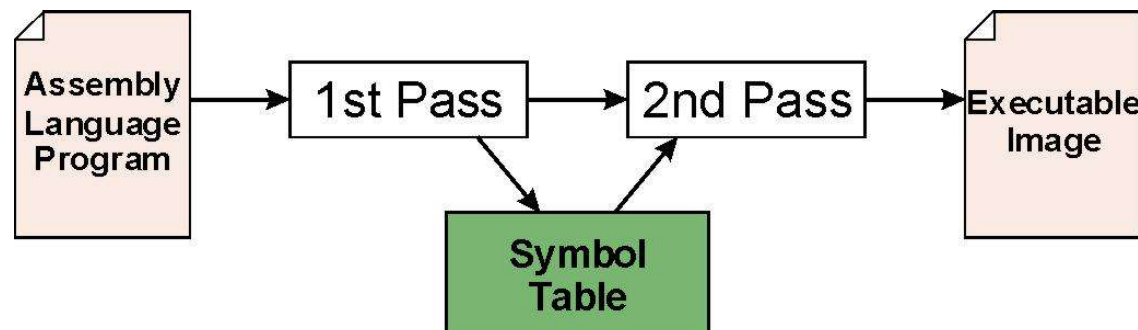
- The assembler translates assembly language programs (.asm) into the machine language of the ISA (.obj).
- There is a 1-to-1 correspondence between assembly language instructions and instructions in the final machine language.

First Pass:

- find all labels and their corresponding addresses
this information is stored in the symbol table

Second Pass:

- convert instructions to machine language, using information from symbol table



Introduction to Assembly Language Programming



1. Introduction

- What is assembly programming
- Why and when to use assembly programming

2. Assembly Program Basics

- Stages in assembly program development
- Assembly Language Syntax

3. Assembler Directives

- Select assembler sections (.sect, .text, .bss, .usect).
- Define values for memory locations (.byte, .word, .string).
- Create symbol table entries (.equ, .set).
- Define library references and definitions (.global, .ref, .def)
- Specify the end of program (.end)
- Assembly Process

4. Information on Executable Instructions

Executable Instruction Information

Executable Instructions

- Executable instructions are assembly mnemonic instructions that are assembled into machine codes and executed by the MSP430.
- Information for the assembly instructions can be obtained from:
 - MSP430 Quick Reference
 - MSP430 Instruction Set Summary
- Information includes:
 - Machine code translation
 - Instruction operation and description
 - Impact on the status bits
 - Length and clock-cycle information

Executable Instruction Information

Machine Code Translation

- Using the MSP430 Quick Reference

opcode				source	Ad	B/W	As	destination	Two-operand arithmetic
0	1	0	1	source	Ad	B/W	As	destination	ADD Add source to destination

B/W: 0 = 16-bits, 1 = 8-bits

As	Ad	d/s	Register	Syntax	Description
00	0	ds	$n \neq 3$	Rn	Register direct. The operand is the contents of Rn. $A_d=0$
01	1	ds	$n \neq 0, 2, 3$	x(Rn)	Indexed. The operand is in memory at address $Rn+x$.
10	-	s	$n \neq 0, 2, 3$	@Rn	Register indirect. The operand is in memory at the address held in Rn.
11	-	s	$n \neq 0, 2, 3$	@Rn+	Indirect auto-increment. As above, then the register is incremented by 1 or 2.
Addressing modes using R0 (PC)					
01	1	ds	0 (PC)	LABEL	Symbolic. x(PC) The operand is in memory at address $PC+x$.
11	-	s	0 (PC)	#x	Immediate. @PC+ The operand is the next word in the instruction stream.
Addressing modes using R2 (SR) and R3 (CG), special-case decoding					
01	1	ds	2 (SR)	&LABEL	Absolute. The operand is in memory at address x.
10	-	s	2 (SR)	#4	Constant. The operand is the constant 4.
11	-	s	2 (SR)	#8	Constant. The operand is the constant 8.
00	-	s	3 (CG)	#0	Constant. The operand is the constant 0.
01	-	s	3 (CG)	#1	Constant. The operand is the constant 1. There is no index word.
10	-	s	3 (CG)	#2	Constant. The operand is the constant 2.
11	-	s	3 (CG)	#-1	Constant. The operand is the constant -1.

ADD.W R5, R6

Binary: 0101 0101 0000 0110

Hex: 0x5506

Instruction Operation and Description

- Using the MSP430 Quick Reference or Instruction Set Summary

			Status Bits			
			V	N	Z	C
ADC(.B)	dst	dst + C → dst	x	x	x	x
ADD(.B)	src, dst	src + dst → dst	x	x	x	x
ADDC(.B)	src, dst	src + dst + C → dst	x	x	x	x
AND(.B)	src, dst	src .and. dst → dst	0	x	x	x
BIC(.B)	src, dst	.not.src .and. dst → dst	-	-	-	-
BIS(.B)	src, dst	src .or. dst → dst	-	-	-	-
BIT(.B)	src, dst	src .and. dst	0	x	x	x

BIS.W #0100h, R6 ;assume initial R6=0x0001

0000 0001 0000 0000

OR 0000 0000 0000 0001

R6= 0000 0001 0000 0001 = 0x0101

Executable Instruction Information

Impact on status flags

- Using the MSP430 Quick Reference or Instruction Set Summary

			Status Bits			
			V	N	Z	C
ADC(.B)	dst	dst + C → dst	x	x	x	x
ADD(.B)	src, dst	src + dst → dst	x	x	x	x
ADDC(.B)	src, dst	src + dst + C → dst	x	x	x	x
AND(.B)	src, dst	src .and. dst → dst	0	x	x	x
BIC(.B)	src, dst	.not.src .and. dst → dst	-	-	-	-
BIS(.B)	src, dst	src .or. dst → dst	-	-	-	-
BIT(.B)	src, dst	src .and. dst	0	x	x	x

ADD.W #8123h, R6 ;assume initial R6=0x9001

$$\begin{array}{r}
 1000\ 0001\ 0010\ 0011 \\
 +\ 1001\ 0000\ 0000\ 0001 \\
 \hline
 \text{R6} = 0001\ 0001\ 0010\ 0100 =
 \end{array}$$

0x1124

N=0, V=1, C=1, Z=0

Executable Instruction Information

Length and Clock Cycle

- Using the Instruction Set Summary

Address Mode		#of cycles	Length of instruction
As	Ad		
00, Rn	0, Rm	1	1
	0, PC	2	1
00, Rn	1, x(Rm)	4	2
	1, EDE		2
	1, &EDE		2
01, x(Rn) 01, EDE 01, &EDE	0, Rm	3	2
			2
01, x(Rn) 01, EDE 01, &EDE	1, x(Rm) 1, TONI 1, &TONI	6	3
			3
			3

ADD 3(R4),6(R9)

Length = 3 Words, Cycle = 6

WHY 3-WORDS and 6-CYCLES???

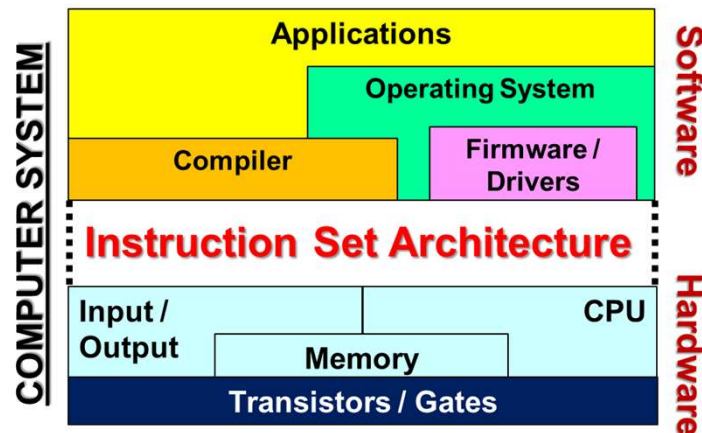
Chpt 4: Introduction to Assembly Language Programming

Summary



LEARNING OUTCOMES OF CHAPTER:

- ✓ Understand the usage of assembly programming
- ✓ Understand the structure of an assembly language program
- ✓ Develop simple programs using the Assembler Directives and Executable Instructions



**NEXT UP:
More on
Executable
Instructions!!!**

Instruction examples – and, bis, xor

Bits 7 6 5 4 3 2 1 0
R10 B' 01010101

N=0, Z=0, V=1, C=0

Initial conditions of byte-sized register R10 and CCR flags

N: Set if MSB of result is set, reset if not set

Z: Set if result is zero, reset otherwise

C: Set if result is not zero, reset otherwise (= .NOT. Zero)

V: Reset

e.g. **and.b #11110000b, R10**

R10 B' 01010000

N=0, Z=0, V=0, C=1

Bit 0 to 3 cleared after execution

e.g. **bis.b #11110000b, R10**

Bis do not change the status bits

R10 B' 11110101

N=0, Z=0, V=1, C=0

Bit 4 to 7 set after execution

e.g. **xor.b #11110000b, R10**

N: Set if MSB of result is set, reset if not set

Z: Set if result is zero, reset otherwise

C: Set if result is not zero, reset otherwise (= .NOT. Zero)

V: Set if both operands are negative

R10 B' 10100101

N=1, Z=0, V=0, C=1

Bit 4 to 7 inverted after execution

Example for Section

```

        .data
Tmp     .byte    8, 9, 10        ; stored in data section
        .data    10             ; stored in data section

        .text
Main    mov.b    R5, R6          ; stored in text section
        .bss     sym,10         ; stored in bss section
        mov      R6,R7          ; stored in text section
    
```